

CO2- LAB EXPERIMENTS

1. Python Program – Map Coloring using CSP

Code

```
colors = ["Red", "Green", "Blue"]
```

```
neighbors = {  
    "A": ["B", "C"],  
    "B": ["A", "C", "D"],  
    "C": ["A", "B", "D"],  
    "D": ["B", "C"]  
}
```

```
def is_valid(region, color, assignment):  
    for neighbor in neighbors[region]:  
        if neighbor in assignment and assignment[neighbor] == color:  
            return False  
    return True
```

```
def solve(assignment):  
    if len(assignment) == len(neighbors):  
        return assignment
```

```
    region = next(r for r in neighbors if r not in assignment)
```

```
    for color in colors:  
        if is_valid(region, color, assignment):  
            assignment[region] = color
```

```

        result = solve(assignment)
        if result:
            return result

        del assignment[region]

    return None

solution = solve({})
print("Map Coloring Solution:")
for region, color in solution.items():
    print(region, ":", color)

```

Input

No input required.

Output

Map Coloring Solution:

A : Red

B : Green

C : Blue

D : Red

2. Python Program – Tic Tac Toe

Code

```

board = [" "] * 9

def display():
    print(board[0], "|", board[1], "|", board[2])
    print("---+---+---")

```

```
print(board[3], "|", board[4], "|", board[5])
```

```
print("---+---+---")
```

```
print(board[6], "|", board[7], "|", board[8])
```

```
def check_winner(player):
```

```
    wins = [
```

```
        (0,1,2), (3,4,5), (6,7,8),
```

```
        (0,3,6), (1,4,7), (2,5,8),
```

```
        (0,4,8), (2,4,6)
```

```
    ]
```

```
    return any(all(board[i] == player for i in w) for w in wins)
```

```
for turn in range(9):
```

```
    display()
```

```
    player = "X" if turn % 2 == 0 else "O"
```

```
    position = int(input(player + " enter position (1-9): ")) - 1
```

```
    if board[position] != " ":
```

```
        print("Position already occupied!")
```

```
        continue
```

```
    board[position] = player
```

```
    if check_winner(player):
```

```
        display()
```

```
        print(player, "wins!")
```

```
        break
```

```
else:
```

```
display()
print("It's a draw!")
```

Input

```
X enter position (1-9): 1
O enter position (1-9): 2
X enter position (1-9): 5
O enter position (1-9): 3
X enter position (1-9): 9
O enter position (1-9): 4
X enter position (1-9): 7
```

Output

```
X wins!
```

3. Python Program – Minimax Algorithm for Gaming

Code

```
def minimax(depth, nodeIndex, maximizingPlayer, values, height):
    if depth == height:
        return values[nodeIndex]

    if maximizingPlayer:
        return max(
            minimax(depth + 1, nodeIndex * 2, False, values, height),
            minimax(depth + 1, nodeIndex * 2 + 1, False, values, height)
        )
    else:
        return min(
            minimax(depth + 1, nodeIndex * 2, True, values, height),
            minimax(depth + 1, nodeIndex * 2 + 1, True, values, height)
```

)

```
values = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
height = 3
```

```
result = minimax(0, 0, True, values, height)
```

```
print("Leaf Values:", values)
```

```
print("Best Value using Minimax:", result)
```

Input

Leaf Values:

3 5 2 9 12 5 23 23

Output

Leaf Values: [3, 5, 2, 9, 12, 5, 23, 23]

Best Value using Minimax: 12

4. Python Program – Alpha-Beta Pruning

Code

```
def alphabeta(depth, nodeIndex, maximizingPlayer, values, alpha, beta, height):
```

```
    if depth == height:
```

```
        return values[nodeIndex]
```

```
    if maximizingPlayer:
```

```
        best = -9999
```

```
    for i in range(2):
```

```
        value = alphabeta(
```

```
            depth + 1, nodeIndex * 2 + i,
```

```

        False, values, alpha, beta, height
    )
    best = max(best, value)
    alpha = max(alpha, best)

    if beta <= alpha:
        break

    return best

else:
    best = 9999

for i in range(2):
    value = alphabeta(
        depth + 1, nodeIndex * 2 + i,
        True, values, alpha, beta, height
    )
    best = min(best, value)
    beta = min(beta, best)

    if beta <= alpha:
        break

    return best

```

```

values = [3, 5, 2, 9, 12, 5, 23, 23]

```

```

height = 3

```

```
result = alphabeta(0, 0, True, values, -9999, 9999, height)
```

```
print("Leaf Values:", values)
```

```
print("Best Value using Alpha-Beta Pruning:", result)
```

Input

Leaf Values:

3 5 2 9 12 5 23 23

Output

Leaf Values: [3, 5, 2, 9, 12, 5, 23, 23]

Best Value using Alpha-Beta Pruning: 12

5. Python Program – Decision Tree

Code

```
from sklearn.tree import DecisionTreeClassifier
```

```
X = [[1], [2], [3], [4], [5], [6]]
```

```
y = ["No", "No", "No", "Yes", "Yes", "Yes"]
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X, y)
```

```
age = int(input("Enter age: "))
```

```
prediction = model.predict([[age]])
```

```
print("Prediction:", prediction[0])
```

Input

Enter age: 5

Output

Prediction: Yes

6. Python Program – Feed Forward Neural Network

Code

```
from sklearn.neural_network import MLPClassifier
```

```
X = [[0,0], [0,1], [1,0], [1,1]]
```

```
y = [0, 1, 1, 0]
```

```
model = MLPClassifier(  
    hidden_layer_sizes=(4,),  
    activation="relu",  
    solver="lbfgs",  
    max_iter=1000,  
    random_state=1  
)
```

```
model.fit(X, y)
```

```
a = int(input("Enter first input (0/1): "))
```

```
b = int(input("Enter second input (0/1): "))
```

```
prediction = model.predict([[a, b]])
```

```
print("Predicted Output:", prediction[0])
```

Input

```
Enter first input (0/1): 1
```

Enter second input (0/1): 0

Output

Predicted Output: 1

7. Prolog Program – Sum of Integers from 1 to N

Code

```
sum_n(0, 0).
```

```
sum_n(N, Sum) :-
```

```
    N > 0,
```

```
    N1 is N - 1,
```

```
    sum_n(N1, S),
```

```
    Sum is N + S.
```

Input

```
?- sum_n(10, Sum).
```

Output

```
Sum = 55.
```

8. Prolog Program – Database with Name and DOB

Code

```
person(john, 10, 5, 2000).
```

```
person(alice, 15, 8, 2001).
```

```
person(bob, 20, 3, 1999).
```

```
person(mary, 25, 12, 2002).
```

```
show_person(Name, Day, Month, Year) :-
```

```
    person(Name, Day, Month, Year).
```

Input

?- show_person(alice, Day, Month, Year).

Output

Day = 15,

Month = 8,

Year = 2001.

9. Prolog Program – STUDENT-TEACHER-SUB-CODE

Code

student(john, cse).

student(alice, ece).

student(bob, cse).

teacher(prof_raj, cse).

teacher(prof_anita, ece).

subject(cse101, programming).

subject(cse102, data_structures).

subject(ece101, electronics).

teaches(prof_raj, cse101).

teaches(prof_raj, cse102).

teaches(prof_anita, ece101).

studies(john, cse101).

studies(john, cse102).

studies(alice, ece101).

studies(bob, cse101).

Input

?- student(john, Department).

?- teacher(prof_raj, Department).

?- teaches(prof_raj, Subject).

?- studies(john, Subject).

Output

Department = cse.

Department = cse.

Subject = cse101 ;

Subject = cse102.

Subject = cse101 ;

Subject = cse102.

10. Prolog Program – PLANETS Database

Code

planet(mercury, 1, terrestrial).

planet(venus, 2, terrestrial).

planet(earth, 3, terrestrial).

planet(mars, 4, terrestrial).

planet(jupiter, 5, gas_giant).

planet(saturn, 6, gas_giant).

planet(uranus, 7, ice_giant).

planet(neptune, 8, ice_giant).

planet_name(Name) :-

planet(Name, _, _).

planet_type(Name, Type) :-

planet(Name, _, Type).

Input

?- planet(earth, Position, Type).

?- planet_type(jupiter, Type).

?- planet_name(Name).

Output

Position = 3,

Type = terrestrial.

Type = gas_giant.

Name = mercury ;

Name = venus ;

Name = earth ;

Name = mars ;

Name = jupiter ;

Name = saturn ;

Name = uranus ;

Name = neptune.